

Humanoid Academy

Tag 3: Low Level
Steuerung, Hände und
Endeffektorsteuerung



Zeitplan

Tag 3

Tag 3

08:30 Finden der Teilnehmer

09:00 Schulungsblock 1

10:30 Kaffee- und Teepause

10:45 Schulungsblock 2

12:00 Mittagessen

13:00 Schulungsblock 3

14:30 Kaffee- und Teepause mit Snacks

15:00 Schulungsblock 4

17:00 Ende Tag 3



Tagesziele

- LowLevel joint control sicher anwenden
- Unterschied arm_sdk vs. LowCmd / dev_mode verstehen
- q , dq , Kp , Kd und τ korrekt einordnen
- Safety ramping: Increment, Clamp und Ramp verwenden
- `g1.release_arms()`, `g1.engage_arms()`, `g1.move_ll_joint()` testen
- Dex3: open/close, release_fingers und tactile pressures auslesen
- Teach & Repeat mit Beispiel: Kaffee-/Sodadose greifen
- Inverse Kinematics mit `ik_move_ee` und `ik_pose_cli_v3` nutzen



Overview Hand & Armvarianten

DFX Dexterous Hand

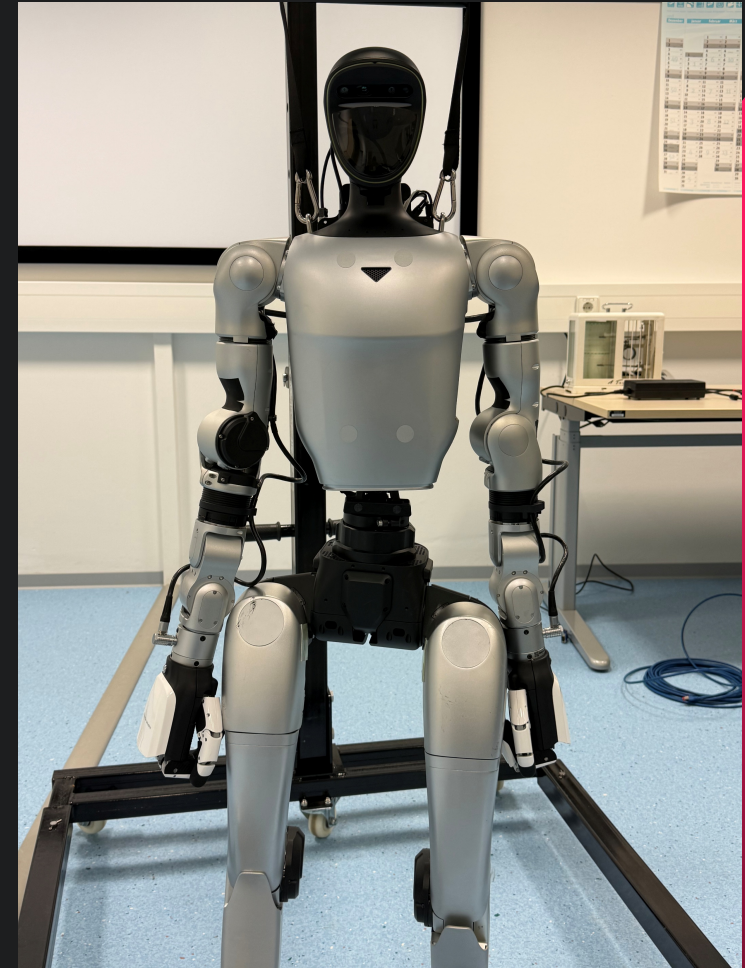


G1 Basic (Tanzen)

⑨ Dance Mode	R1 + B
└ Kung Fu	R1 + SELECT
└ Dance 1	R1 + 
└ Dance 2	R1 + 
└ Dance 3	R1 + 
└ Roll Up	R1 + 



G1 FÜNF FINGER



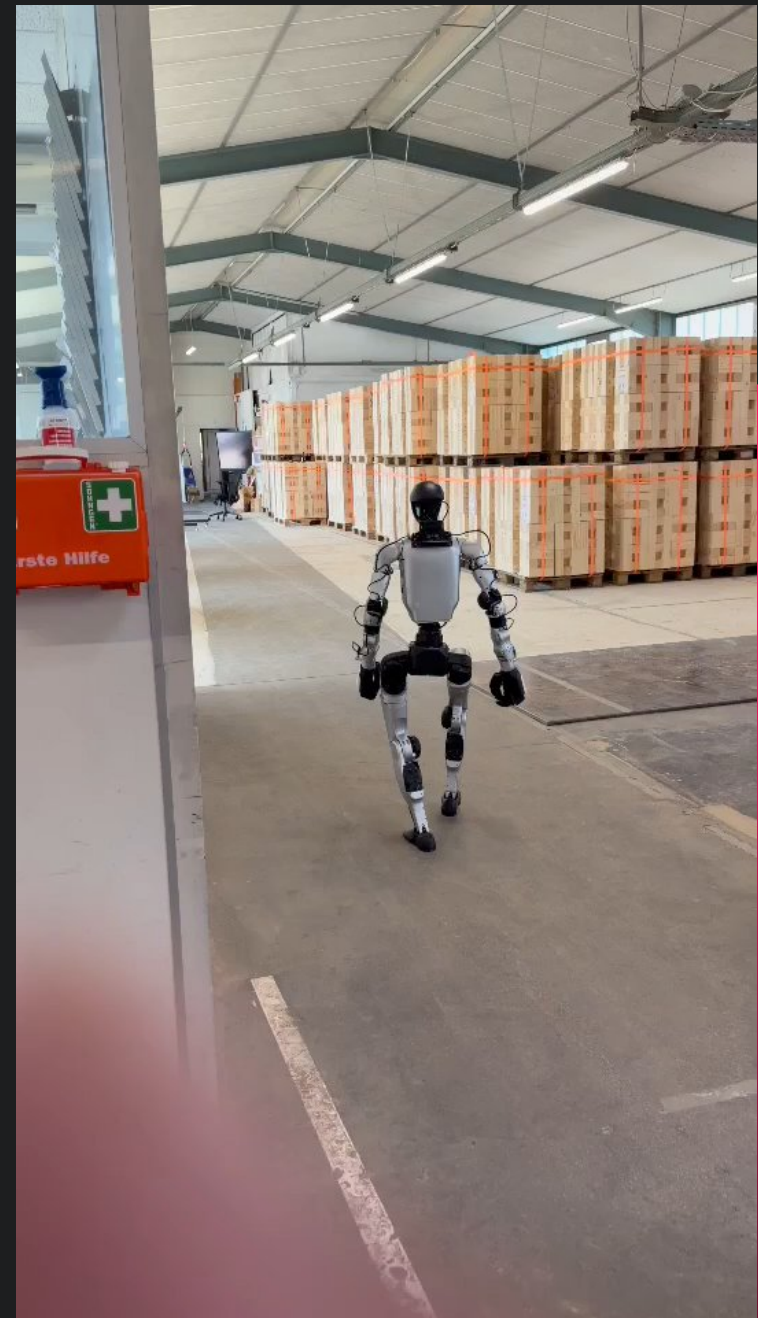
G1 FÜNF FINGER







LocoClient Stabilität mit DFX Hände (Running Mode)



LocoClient Stabilität mit DFX Hände (Regular Mode)



Low Level Steuerung

- LowLevel joint control
- Arm SDK vs. LowCmd / dev_mode
- q , dq , K_p , K_d , τ
- Safety ramping: Increment $\rightarrow$ Clamp $\rightarrow$ Ramp
- Nur eine aktive Kontrollquelle pro Arm verwenden



LowLevel Joint Control

- Konzept
- Ziel: einzelne Gelenke direkt und kontrolliert bewegen
- arm_sdk
- High-Level Armsteuerung über SDK
- Für Gesten, Posen und reguläre Armbewegungen
- LowCmd / dev_mode
- Direkte Gelenkbefehle an q , dq , K_p , K_d , τ
- Nur mit Begrenzung und Rampen verwenden



q , dq , K_p , K_d und τ

- q – Gelenkposition
- Ziel- oder Ist-Winkel des Gelenks in rad
- dq – Gelenkgeschwindigkeit
- Änderung der Gelenkposition in rad/s
- K_p – Positionsverstärkung
- Wie stark das Gelenk dem Positionsfehler folgt
- K_d – Dämpfungsverstärkung
- Reduziert Schwingen und schnelle Bewegungen
- τ – Gelenkmoment
- Drehmomentbefehl oder gemessenes Moment in Nm



Increment, Clamp, Ramp – Never Jump

- Bei der Low-Level- und IK-Regelung sollten Zielwertänderungen schrittweise erfolgen.
- Inkrement
 - Um einen kleinen Betrag verschieben: Zielwert += Delta
- Clamp
 - Den Zielwert innerhalb sicherer Grenzen halten: Zielwert = Clamp(Zielwert, Min_Wert, Max_Wert)
- Ramp
 - Schrittweise vom aktuellen Wert zum Zielwert übergehen. Aktuell
 - ↓
 - Schritt
 - ↓
 - Schritt
 - ↓
 - Schritt
 - ↓
 - Zielwert



G1 Wrapper– Arm & Joint

- `g1.release_arms()`
- Arme freigeben und manuell führen
- `g1.engage_arms()`
- Armregelung wieder aktivieren
- `g1.move_ll_joint(joint, delta_or_target)`
- Einzelnes Gelenk sicher bewegen
- Beispiel: right shoulder roll um $+0.3$ rad



Visualisierung 1: joint_control_dashboard

- Live-Anzeige für Low-Level Joint Control
- q : aktuelle Gelenkposition
- $\dot{q}$: Gelenkgeschwindigkeit
- τ : Drehmoment / Moment-Rückmeldung
- Ziel- q und Ramp-Fortschritt sichtbar machen
- Nutzen: sichere Tests einzelner Gelenke ohne Sprungbewegungen



Overview Hand & Armvarianten

Dex3-1

Three-fingered dexterous hand Dex3-1 Electrical Parameter	
Rendering image	
Operating voltage	12-58V
Range of Perception	10g-2500g
Degrees of freedom	Total degrees of freedom: 7 1.The thumb has 3 active degrees of freedom; 2.the index finger has 2 active degrees of freedom; 3.the middle finger has 2 active degrees of freedom.
Angle of Joint	Thumb: 0°~+100°, -35°~+60°, -60°~+60°; Index Finger and Middle Finger: 0°~+90°, 0°~+100°.
Number of Array Sensors	9

G1 3 FINGER



G1 3 FINGER



G1 Wrapper – Dex3

- `g1.open_dex3_hand(hand)`
- Dex3 Hand schrittweise öffnen
- `g1.close_dex3_hand(hand)`
- Dex3 Hand schrittweise schließen
- `g1.release_fingers(hand)`
- Finger freigeben / compliant schalten
- `g1.get_dex3_tactile_pressures(hand)`
- Taktile Druckwerte auslesen



Dex3 Methods

- Die Dex3-Funktionen werden als wiederverwendbare Services genutzt.
- Wichtige Funktionen:
 - `g1.open_dex3_hand(hand)`
 - `g1.close_dex3_hand(hand)`
 - `g1.release_fingers(hand)`
 - `g1.get_dex3_tactile_pressures(hand)`
- Ziel: Handsteuerung, taktile Rückmeldung und Greifen in einfachen Programmen kombinieren.

Architecture

Application



Dex3 Service



Hand Command



Dex3

This makes later pick-and-place tasks easier to implement.

Aufgabe 1: LowLevel + Dex3 Basischeck

- 1. Rechte Schulter-Roll um +0.3 rad bewegen
- `g1.move_ll_joint("right_shoulder_roll", +0.3)`
- 2. Arme freigeben
- `g1.release_arms()`
- 3. Arme wieder aktivieren
- `g1.engage_arms()`
- 4. Tactile Pressures auslesen
- `g1.get_dex3_tactile_pressures("right")`
- 5. Dex3 Hand öffnen und schließen
- `g1.open_dex3_hand("right") /`
`g1.close_dex3_hand("right")`
- 6. Finger freigeben
- `g1.release_fingers("right")`



Teach & Repeat – Beispiel

- teach
 - 1. g1.release_arms() und g1.release_fingers() ausführen
 - 2. Arm manuell zum Objekt führen
 - 3. Cup/Can greifen und wichtige Posen speichern
- repeat
 - 1. g1.engage_arms() aktivieren
 - 2. gespeicherte Gelenksequenz interpoliert abspielen
 - 3. Objekt greifen, halten und wieder freigeben



Grundlagen der Armtätigkeit

Gelenke

```
TOPIC_ARM = "rt/arm_sdk"
TOPIC_HAND_RIGHT = "rt/dex3/right/cmd"

G1_NUM_MOTOR = 29
ARM_ENABLE_IDX = 29 # enable arm_sdk when q = 1 (per Unitree docs)

class G1JointIndex:
    WaistYaw = 12
    WaistRoll = 13
    WaistPitch = 14
    RightShoulderPitch = 22
    RightShoulderRoll = 23
    RightShoulderYaw = 24
    RightElbow = 25
    RightWristRoll = 26
    RightWristPitch = 27
    RightWristYaw = 28

ARM_JOINTS = [
    G1JointIndex.RightShoulderPitch,
    G1JointIndex.RightShoulderRoll,
    G1JointIndex.RightShoulderYaw,
    G1JointIndex.RightElbow,
    G1JointIndex.RightWristRoll,
    G1JointIndex.RightWristPitch,
    G1JointIndex.RightWristYaw,
]
```

Festprogrammierte Greifaufgabe

Steuerung einzelner Finger

```
# Hand joint order as used by unitree_sdk2py dex3 examples.  
# If your hand responds incorrectly, swap index/middle pairs to match your firmware.  
HAND_JOINTS = [  
    "thumb_0",  
    "thumb_1",  
    "thumb_2",  
    "middle_0",  
    "middle_1",  
    "index_0",  
    "index_1",  
]
```

- Alternativ:
 - Robot.hand_open(„right“ or „left“)
 - Robot.hand_close(...)

G1 3-Finger Greifer (Konzept)

- 3 Finger Greifer für stabile und adaptive Greifbewegungen
- Geeignet für Werkzeuge und industrielle Objekte



Greifprinzip

- Kraftschluss (Force Closure)
- Formschluss (Shape Fit)
- Drei Kontaktpunkte für Stabilität



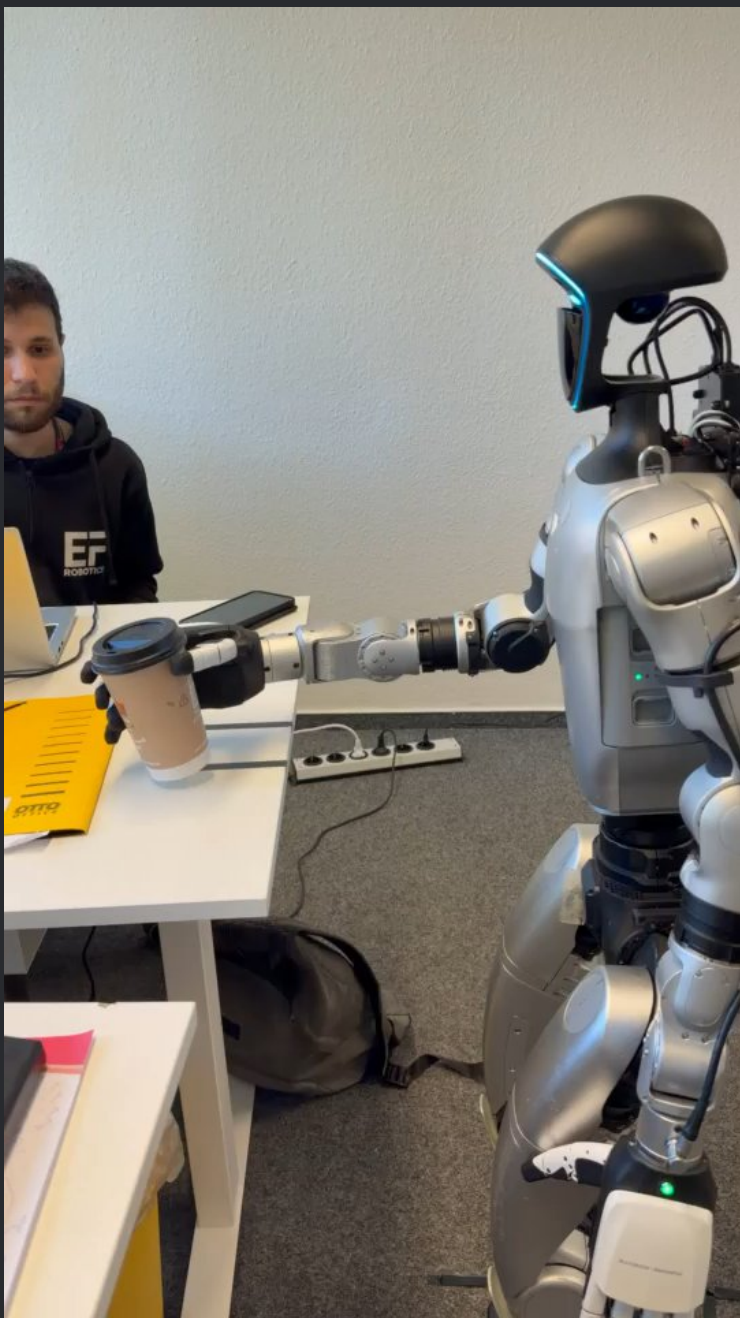
Safe Arm Pose Writing

- Empfohlene Abfolge
- Aktuelle Armposition auslesen
- Zielposition überprüfen
- Maximale Änderung begrenzen
- Interpolation von der aktuellen zur Zielposition
- Zwischenpositionen senden
- Endgültige Gelenkposition überprüfen

`</>` Python

```
write_arm_sdk_pose(target_pose)
```





Aufgabe 2: Teach & Repeat

- Use teach & repeat to grab a coffee cup or soda can.
- Ablauf:
 - 1. `g1.release_arms()` und `g1.release_fingers()`
 - 2. Greifbewegung manuell teachen
 - 3. Posen und Zeitstempel speichern
 - 4. `g1.engage_arms()`
 - 5. Repeat ausführen: greifen → halten → freigeben



Humanoid Implementation Program

- Alle 2 Wochen 1 Std Consulting
- WhatsApp Support
- Zugriff auf alle unsere Kurse
- Zugriff auf unseren Softwarestack



Greifarten

- Pinch Grip (Präzisionsgriff)
- Power Grip (Kraftgriff)
- Adaptive Greifbewegung



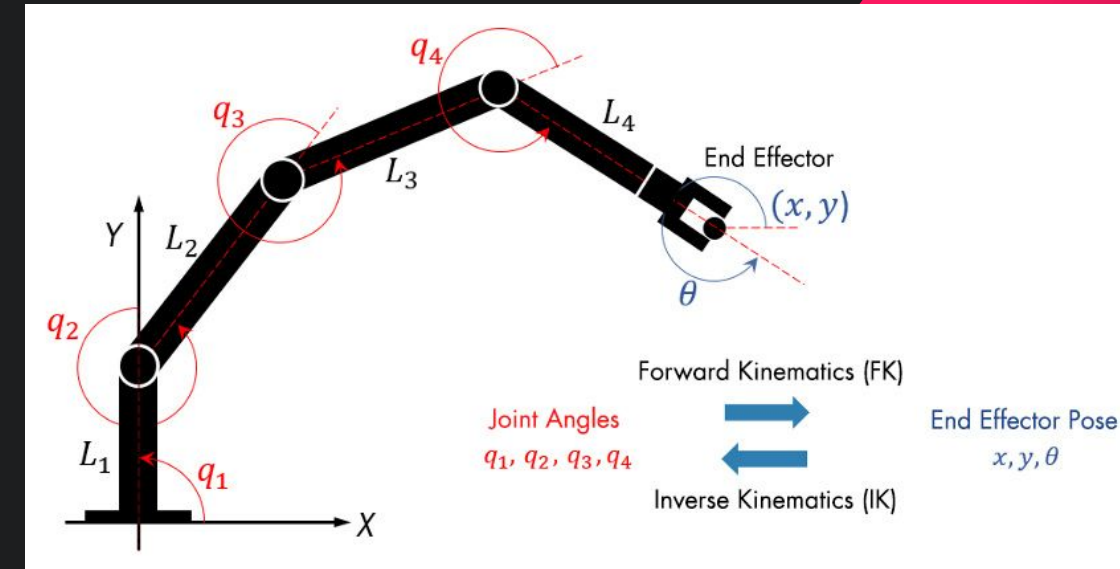
Steuerung des Greifers

- Einzelne Fingersteuerung
- Vordefinierte Greifmuster
- Integration mit Armbewegung



Inverse Kinematics + ik_move_ee

- Inverse Kinematics übersetzt Endeffektor-Ziele in Gelenkbewegungen.
- 6D Pose: x, y, z + yaw, roll, pitch
- ik_move_ee bewegt den Endeffektor schrittweise zur Zielpose.
- Wichtig: kleine Deltas verwenden, clampen und sicher rampen.
- Visualisierung / Tool: ik_pose_cli_v3



Machine Learning

- Regression Modelle:
- Neuronales Netzwerk bekommt aktuelle Gelekpositionen als



Koordination Arm + Greifer

- Arm bewegt sich zur Zielposition
- Greifer schließt sich
- Objekt wird stabil gehalten



Aufgabe: Stabilität testen

- Objekt greifen
- Roboter leicht bewegen
- Überprüfen ob Objekt stabil bleibt



Low-Level Control

- Joint Steuerung
- Arm Bewegung
- Hand Koordination



High-Level Gestures, Teach/Repeat & Pose Interpolation

Drei Ansätze für Armbewegungen

Allgemeine Gesten

- Vordefinierte Roboterbewegungen
- Schnell und einfach auszuführen
- Nützlich für Demonstrationen und Standardposen

Teach & Repeat

- Manuelles Führen des Roboterarms
- Aufzeichnen der Gelenkpositionen
- Wiedergeben der aufgezeichneten Bewegungsbahn
- Poseninterpolation
- Definieren von Start- und Zielpositionen der Gelenke
- Flüssige Interpolation zwischen diesen Positionen
- Ermöglicht kontrollierte und wiederholbare Armbewegungen
- Ziel: Wählen Sie je nach Anwendung die richtige Steuerungsmethode aus.



One Arm Control Path – Never Both at Once

The G1 arm should have **one active command source at a time**.

High-Level Control



Arm SDK



Motors

Low-Level Control



LowCmd



Motors



Arm SDK vs. LowCmd / dev_mode

- Arm SDK
 - High-Level Armsteuerung mit interner Sicherheitslogik
 - Geeignet für Gesten, Standardposen und Teach/Repeat
- LowCmd / dev_mode
 - Direkte Low-Level Gelenkkommandos
 - q , dq , K_p , K_d und τ werden explizit gesetzt
 - Nur mit Clamp, Interpolation und Sicherheitsrampe verwenden



Low-Level Joint Control + Live Visualizer

- **Goal**
- Control individual G1 joints directly and simultaneously observe their state.

```
LowCmd Publisher  
↓  
G1 Joint Controller  
↓  
Robot  
↓  
LowState  
↓  
Live Visualizer
```

This allows direct observation of how a joint responds to a command.

Read:

```
q  
dq  
tau
```

Control:

```
Target q  
Kp  
Kd
```

G1 – 29 Joint Index / Gain Table

- Bei der Low-Level-Steuerung wird jedes Gelenk über seinen Gelenkindex angesprochen.
- Die Konfiguration sollte eine Tabelle enthalten, die folgende Angaben umfasst:
- Gelenkname
- Gelenkindex
- Kp
- Kd
- Minimalposition
- Maximalposition

Example structure:

```
</> Python

joint_config = {
    "right_shoulder_pitch": {
        "index": ...,
        "kp": ...,
        "kd": ...
    }
}
```

LowCmdPublisher + move_R_joint + Live Visualizer

- LowCmdPublisher + move_R_joint + Live Visualizer
- Erstellen Sie drei Komponenten:
 - 1. LowCmdPublisher
 - Veröffentlicht Low-Level-Befehle an den Roboter.
 - 2. move_R_joint()
 - Bewegt ein ausgewähltes Gelenk des rechten Arms sicher in Richtung einer Zielposition.
 - 3. Live Visualizer
 - q
 - dq
 - tau
 - Ziel q



Safe Low-Level Move

- `def safe_move_ll_joint(joint, delta):`
 - `current = g1.read_joint_position(joint)`
 - `target = clamp(current + delta, joint_min, joint_max)`
- `for q in ramp(current, target, step=0.01):`
- `g1.move_ll_joint(joint, q)`
- Task example:
- `safe_move_ll_joint("right_shoulder_roll", 0.3)`



Dex3 – Dual Gripper Control

- Goal
- Create basic control functions for both Dex3 hands.

Functions:

`<>` Python

```
open_hand("left")  
open_hand("right")  
  
close_hand("left")  
close_hand("right")
```

Recommended sequence

```
Read current finger positions  
↓  
Determine baseline  
↓  
Gradually close  
↓  
Detect target / threshold  
↓  
Hold
```

Both hands should be independently controllable.

Dex3 HandCmd Message and Joint Range

- Die Dex3-Steuerung erfordert Befehle für einzelne Handgelenke.
- Ein Handbefehl sollte die erforderlichen Gelenkzielwerte und Steuerparameter enthalten.
- Konzeptionell:
- HandCmd
 - — Gelenk 1
 - — Gelenk 2
 - — Gelenk 3
 - — ...
 - — Gelenk n
- Wichtig
- Jedes Fingergelenk hat einen gültigen Bewegungsbereich.
- Vor der Veröffentlichung:
- `target = clamp(`
- `target,`
- `joint_min,`
- `joint_max`
- `)`



Calibrate the Threshold – Don't Copy One

- Eine Greifschwelle sollte nicht einfach von einem anderen Roboter oder einer anderen Hand kopiert werden.
- Mechanische Unterschiede können folgende Ursachen haben:
- Einbau der Hand
- Ausrichtung der Finger
- Geometrie des Objekts
- Sensortoleranz
- Mechanischer Verschleiß
- Unterschiedliche Handmodule

Calibration procedure

```
graph TD; A[Open Hand] --> B[Record Baseline]; B --> C[Close Slowly]; C --> D[Observe Joint Response]; D --> E[Determine Threshold]; E --> F[Store Calibration];
```

Calibrate each hand before relying on a gripping threshold.

- Ausgangsposition → Allmähliches Schließen
→ Allmähliches Öffnen
- Ein sicherer Greifvorgang sollte kontrollierte, schrittweise Bewegungen beinhalten.
- 1. Ausgangsposition
- Die anfängliche Position der offenen Hand erfassen.
- 2. Allmähliches Schließen
- 3. Allmähliches Öffnen
- Die Abfolge umkehren, anstatt sofort die vollständig geöffnete Position zu veranlassen.
- Vorteile
- Flüssigere Fingerbewegung
- Einfachere Fehlerbehebung
- Bessere Handhabung von Objekten
- Weniger plötzliche Bewegungen



Gradual Grip – Example Structure

`</>` Python

```
def gradual_close(hand):  
  
    baseline = read_hand_state(hand)  
  
    while not grip_reached():  
  
        target = calculate_next_step()  
  
        target = clamp_joint_limits(target)  
  
        send_hand_command(  
            hand,  
            target  
        )  
  
        sleep(dt)
```

Opening uses the same principle in reverse.



Wichtige ROS2 Topics

- /map → Generated map
- /odom → Robot movement
- /tf → Transform tree
- /scan → LiDAR data
- /camera/depth → Depth

```
/clicked_point  
/cmd_vel  
/goal_pose  
/initialpose  
/joint_states  
/parameter_events  
/robot_description  
/rosout  
/tf  
/tf_static
```

Workspace Struktur

- docs, scripts, modules
- G1 sport mode ROS
- simulation



Setup

❏ Bash

```
nano ~/.bashrc
```



❏ Bash

```
source ~/.bashrc
```

❏ Bash

```
# Source ROS 2 Foxy
```

```
source /opt/ros/foxy/setup.bash
```

```
# Source your workspace (change path if needed)
```

```
source ~/foxy_ws/install/setup.bash
```

❏ Bash



```
colcon build
```

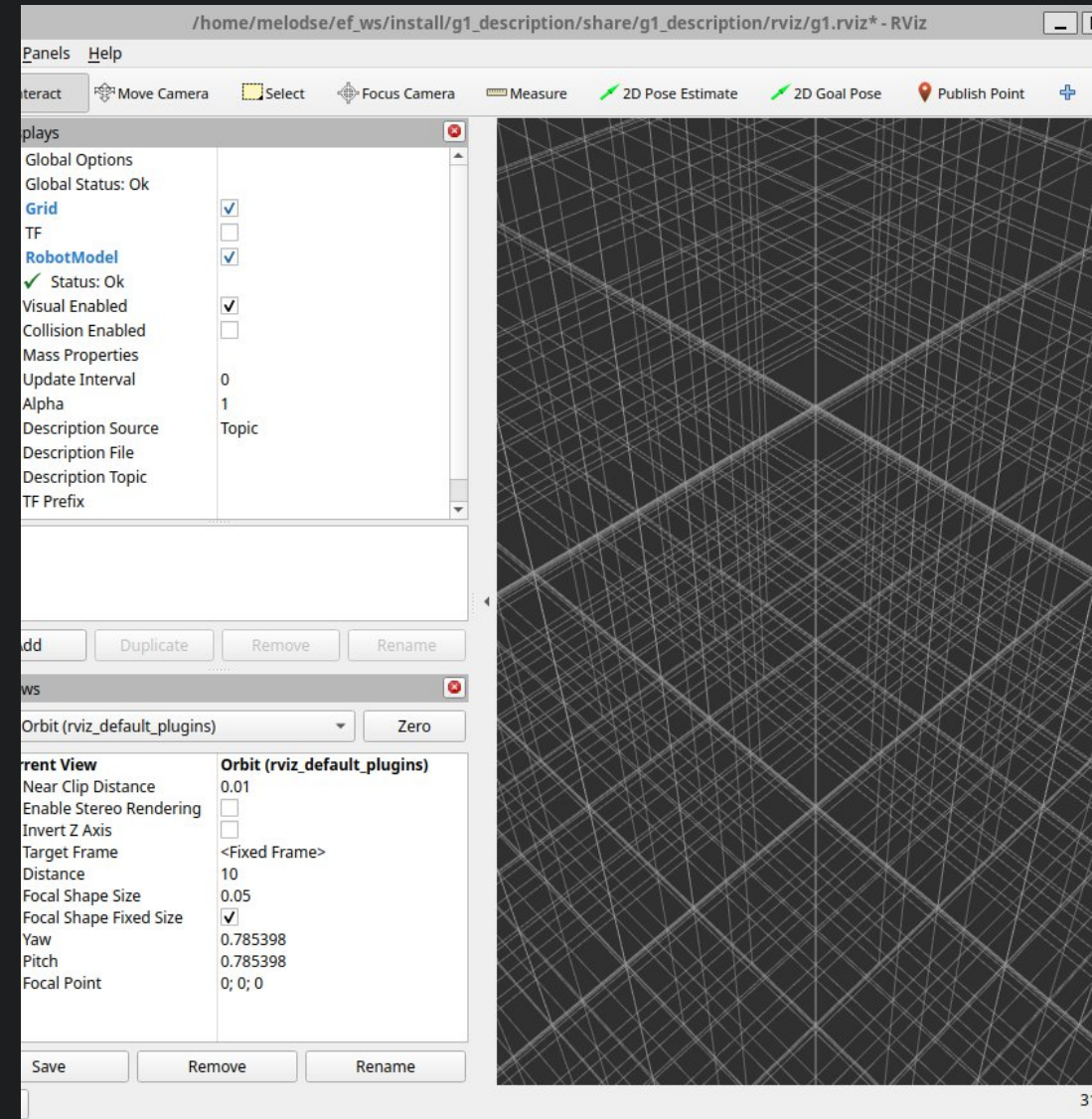
❏ Bash



```
colcon build --packages-select package_name
```

RViz Setup (Schritt für Schritt)

- 1. Launch robot / simulation
- 2. Start SLAM node
- 3. Open RViz
- 4. Add displays



Topics:

Bash



```
ros2 topic list
```

If not correct:

Bash



```
ros2 daemon stop  
ros2 daemon start  
ros2 topic list
```

Still not working:

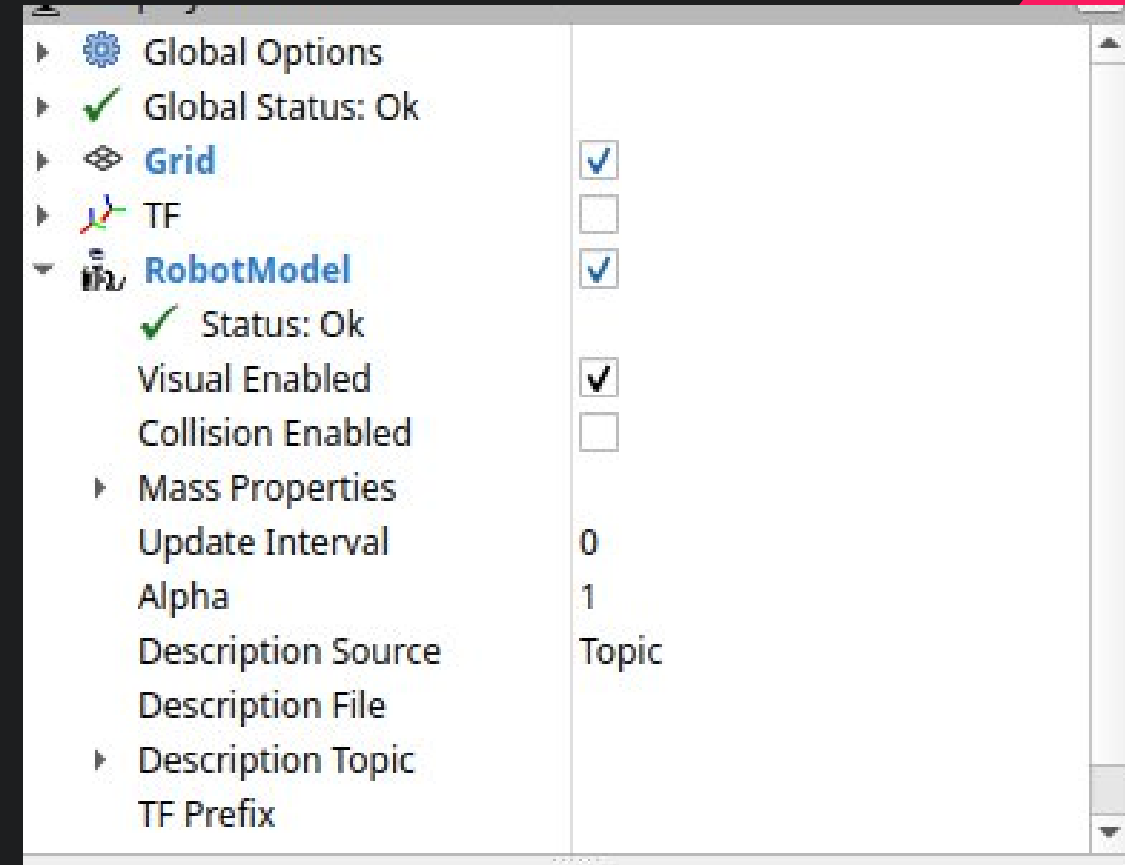
Bash



```
echo $ROS_DOMAIN_ID
```

RViz Anzeigen hinzufügen

- Map → /map
- TF → transforms
- LaserScan → /scan
- RobotModel → URDF



RobotModel

- ✓ Status: Ok
- Visual Enabled ☒
- Collision Enabled ☐
- Mass Properties
 - Update Interval 0
 - Alpha 1
 - Description Source Topic
 - Description File
- Description Topic **/robot_description**
- TF Prefix
- Links

Description Topic
Topic where filepath to urdf is published.

Add Duplicate Remove Rename

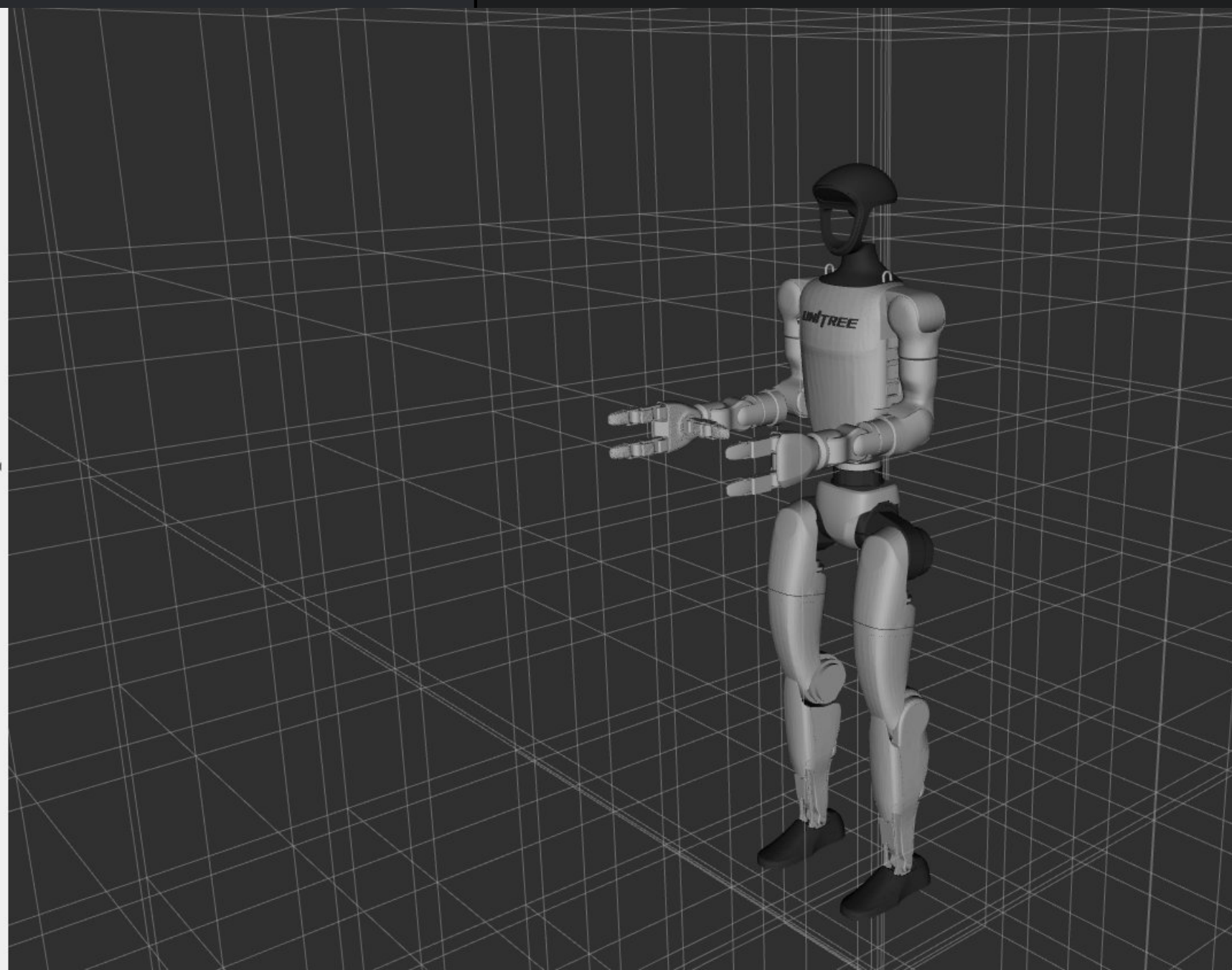
Views

Type: Orbit (rviz_default_plugins) Zero

Current View

- Near Clip Distance 0.01
- Enable Stereo Rendering ☐
- Invert Z Axis ☐
- Target Frame <Fixed Frame>
- Distance 2.51916
- Focal Shape Size 0.05
- Focal Shape Fixed Size ☒
- Yaw 0.815398
- Pitch 0.255398
- Focal Point 0; 0; 0

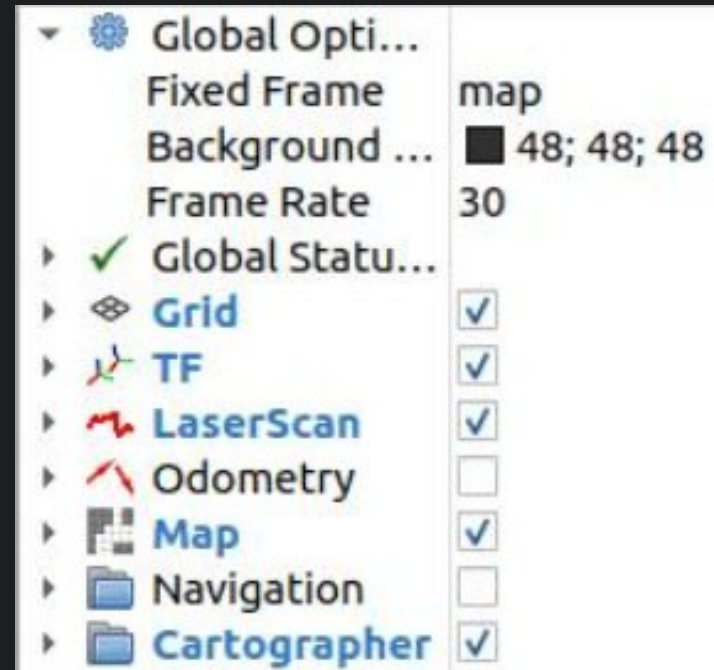
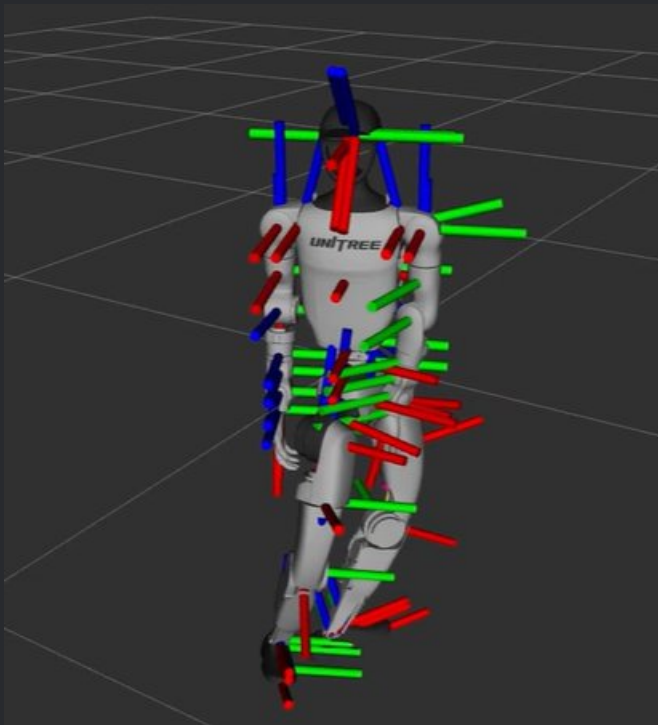
Orbit (rviz_default_plugins)



G1 RViz

Simulationsablauf

- Sensor Data → SLAM → Map
- Odometry → TF → Pose
- Everything visualized in RViz



Tanzen G1 simulation

Step 1: Set up a Conda virtual environment

```
conda create -n retarget python=3.10
```

```
conda activate retarget
```

Step 2: Install dependencies

```
conda install pinocchio -c conda-forge
```

```
pip install numpy rerun-sdk==0.22.0 trimesh joblib
```

Step 3: Run the script with parameters:

```
python rerun_visualize.py --file_name dance1_subject2 --robot_type [g1_23dof|g1_29dof|h1|h1_2]
```



Bonus Aufgabe : ik_pose_cli_v3

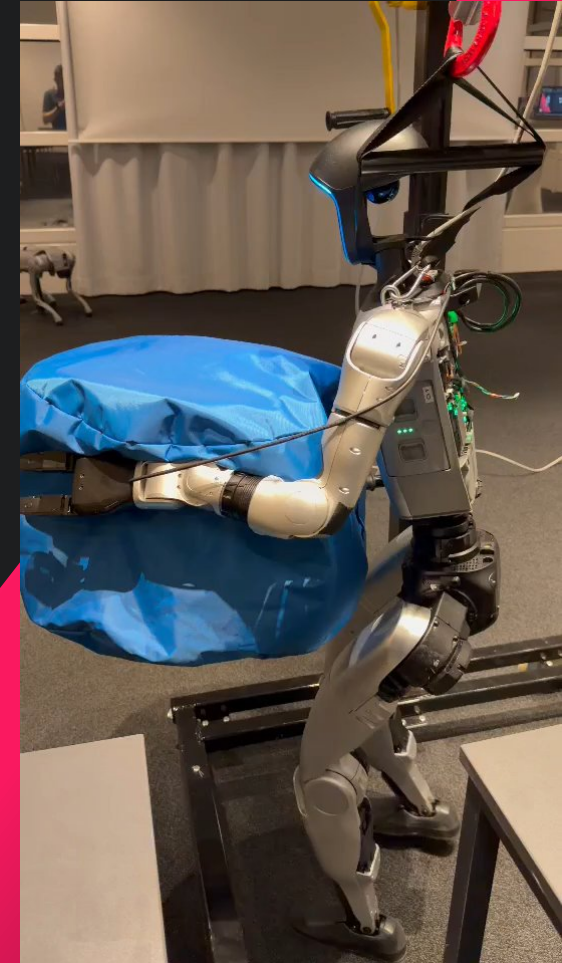
Use ik_pose_cli_v3 to save an arm motion sequence.

Goal options:

- record and save arm movement
- replay the saved sequence
- example: grab a cup / soda can

Workflow:

select arm → ik_move_ee → save key poses
→ repeat



Aufgabe 4: Codex

Dashboard

Create a dashboard-like UI using Codex.

Example prompt:

Build a G1 end-effector teleoperation dashboard with buttons for:

- select right / left arm
- release_arms and engage_arms
- open / close Dex3 hand and release_fingers
- increase / decrease x, y, z
- increase / decrease yaw, roll, pitch
- call ik_move_ee safely with clamp and ramp
- display joint_control_dashboard values



Sensor Daten + get_rgbd_data

- Kamera: RGB-D Daten erfassen
- LiDAR: Umgebung / Hindernisse
- IMU: Lage und Bewegung
- Bonus-Funktion:
- `g1.get_rgbd_data()`
- Nutzen: RGB + Depth als Basis für Objektlokalisierung und Greifaufgaben.



Bonus Aufgabe: `get_rgbd` data

Goal: read RGB-D data from the robot camera interface.

Use case examples:

- show live RGB image
- inspect depth values near the object
- support cup/can localization before IK motion

Minimal workflow:

1. `data = g1.get_rgbd_data()`
2. display RGB frame
3. check depth at target region
4. use depth result as input for grasp planning

